CE/CS/SE 3354 · Software Engineering · Fall 2026

Project Milestone P2

Analysis and High-Level Design

[Project title]

Team	Team [NN], section [.003 / .005 / .010]
Document version	1.0 (baseline for P2; grows from your P1 version 0.x)
Date	
Repository URL	https://github.com/[owner]/[repository]
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	[one member only]

Team member	NetID	GitHub username

HOW TO USE THIS TEMPLATE

This is not a new document. P2 continues the document you baselined at P1. Copy your P1 sections 1 to 3 into this file (updated under change control), then fill in the new sections 4 to 9. Your P1 project plan, open questions and baseline move to the end and are renumbered:

P1 section 4 (project plan) becomes section 10. P1 section 5 (open questions) becomes 11. P1 section 6 (baseline) becomes 12.

Delete every guidance, example and warning box (blue, amber and red) and the two marking pages at the end before you submit. They are instructions, not content. Boxes or placeholders such as "[TBD]", "PR #[n]" or "[Member 3]" left in the submitted document lose document-quality points.

Length: sections 4 to 9 should add roughly 12 to 18 pages, diagrams included. Every section below carries a budget. A longer document is not a better one.

Diagrams: draw them in any tool (draw.io, Lucid, PlantUML, or a tidy hand drawing that is scanned). Export each as an image for this document and commit the source file to /docs/p2/diagrams. A diagram that cannot be read cannot be credited for its content.

Required structure: using this file is optional, but its section numbering is required, because the rubric refers to it.

Rubric: the marking pages at the end show where the 60 points sit. Read them before you divide the work.

WHAT P2 DOES NOT ASK FOR (DELETE THIS BOX BEFORE SUBMITTING)

P2 is analysis and high-level design. At this milestone we are NOT asking for:

design class diagrams with operations, parameter types or visibility, and design-level sequence diagrams with operation signatures (Chapter 9 detailed design; they enter the document during P3 work)

production code (the only code in P2 is the small architecture spike in section 8)

a test plan (P3, after Chapter 11)

screen mock-ups or a visual design (not graded at this milestone; add them only if they settle an open question)

design patterns from the Gang of Four catalog, ATAM or utility trees (not required; do not spend your time there)

Work placed in the wrong milestone is not credited. Spend the time on sections 4 to 9.

0 Revision History, Contribution Record and Changes Since P1

All three tables below are graded. The contribution record carries the individual 20% of this milestone and must agree with your repository history.

Version	Date	Author	Summary of change
0.x			P1 baseline (copy your last P1 row)
1.0			P2 baseline: analysis and high-level design added

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Evidence means something a reader can open and that existed before the deadline: a commit hash, a pull request, a file in the repository. Issues you own count as indirect evidence; link the commits that close them for full credit.

Commit your own work from your own GitHub account. If one person uploads everyone's text from a shared document, the history shows one author and the other members have no verifiable evidence. A shared drafting document is fine; the final text of each section is committed by its owner.

Cite only what exists. Every commit, pull request, branch and file named in this table is checked. Citing evidence that is not in the repository is an academic-honesty matter, not a formatting slip.

Every member appears with a distinguishable contribution. Two rows with identical wording are marked as vague claims. If a member did not contribute, write that plainly in their row rather than leaving it blank or covering for them.

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01				
CH-02				
CH-03				

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Start with your P1 feedback. Every point raised in your P1 comment should appear here as a change, or with one sentence explaining why you kept the original. This table is where the baseline-and-change-control idea from Chapter 5 becomes real.

Requirement IDs are never reused. A removed requirement stays in section 3 marked "Withdrawn (CH-nn)" so the traceability matrix does not silently lose it.

Budget: half a page.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

CH-02 · QR-02 rewritten as quality-attribute scenario QAS-01 · Why: P1 feedback said "fast enough" had no response measure · Knock-on: ADR-02 now cites QAS-01 · Agreed 30 Oct.

1 to 3 Introduction, Overall Description and Requirements (carried forward)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Paste your P1 sections 1, 2 and 3 here and bring them up to date. Keep their numbering and IDs.

Fix before you build on it: any requirement that is still untestable, any source that still says "pending", any open question your P1 feedback flagged. Analysis and design inherit every ambiguity you leave in section 3.

Use cases: section 3.4 keeps the use-case diagram and the list at actor-goal level. The detailed expansion now lives in section 4; do not copy it twice.

Traceability: replace the P1 matrix in section 3.5 with a one-line pointer to section 9.1, where it is extended.

If nothing in a subsection changed, keep it and say so in one line under its heading ("Unchanged since v0.x").

4 Use-Case Analysis

Chapter 7, first half. Here you take the use cases from section 3.4 and write the most important of them out turn by turn, still with the technology switched off.

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-			
UC-			
UC-			

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
>
> **Analyze at least three use cases,** including the two you specified in full at P1. We recommend that one of them involves an external system or a failure that the actor can see.
>
> Choose by risk and value, not by ease: the use case that carries most of the product, the one that touches most of your quality requirements, the one you are least sure you can build.
>
> Budget: a quarter of a page.

4.2 Expanded use cases

Copy the block below once for each use case in 4.1.

UC-xx [Use case name]

Primary actor	
Supporting actors and external systems	
TUCBW (this use case begins when)	
TUCEW (this use case ends when)	
Preconditions	
Postconditions	
Traces to requirements	FR- , QR- , CON-

Actor input / action	System response
	(0)

Actor input / action	System response
(1)	(2)
(3)	(4)
(5)	(6)

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1				
A2				

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

The rules from Lesson 13, all of which are checked:

Step (0) is what the actor sees before touching anything. It is often skipped, and it is where many missing requirements are found.

One turn per row. An actor action on the left, the system response on the right. Two steps merged into one row become a missing message on your sequence diagram in section 5.

End with an acknowledgment the actor can perceive. No acknowledgment, no TUCEW, no use case.

No background processing. If the actor neither sees it nor waits for it, it belongs to another use case.

Precondition or alternate flow? If it can be false while the system works normally, it is a flow you must handle, not an assumption.

No technology yet. No screen widgets, no database, no framework names. "Taps the green button" and "queries the PostgreSQL table" both fail the analysis check.

Every item the actor column names should already be an attribute in your domain model (5.1). Where it is not, you have found one.

Budget: about one page per use case.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

UC-02 Find an Open Space · Primary actor: Driver · External system: Lot Sensor Network · TUCBW the driver asks for parking near a destination building · TUCEW the driver sees the recommended lot with its number of open spaces.

(0) The system shows the campus lots with the time of the latest occupancy update. (1) The driver chooses a destination building and permit type. (2) The system shows the three nearest lots the permit allows, each with open spaces and walking time. (3) The driver selects a lot. (4) The system shows the lot entrance and confirms the count is no older than 60 seconds.

A1 at step (2): no permitted lot has an open space · the system shows the nearest lot with expected turnover and the visitor option · ends.

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Run the review checklist from Lesson 13 on each expanded use case before anyone draws a diagram, and record what each check found, including any defects you fixed. Report honestly; a clean result is acceptable when it is true. Budget: a quarter of a page.

5 Analysis Models

Chapter 7, second half. What has to exist inside the system for the use cases in section 4 to happen, and how those parts talk to each other.

5.1 Domain model

Insert diagram here, then replace this frame with the image.

Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Conceptual classes only: things and records from the problem domain, with attributes and associations. No operations, no data types, no database tables, no screen classes.

Every item the actor column of section 4.2 mentions is an attribute somewhere in this model. Every association has a multiplicity at both ends.

Budget: one page including the diagram.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

Lot (name, zone, capacity) 1 ── 0..* Space (number, type) ; Space 1 ── 0..* OccupancyReading (time, state) ; Permit (type, valid until) * ── 1..* Lot "is valid in" ; Building (name) * ── * Lot "is near" (walking time).

5.2 Object interaction models

Copy the block below for at least two of the use cases in 4.1; preferably including your riskiest.

5.2.x UC-xx [Use case name]

(a) Non-trivial steps. List the steps from 4.2 that need objects inside the system, and say in one line why each of the others is trivial.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)

(c) Analysis sequence diagram.

Insert diagram here, then replace this frame with the image.

Figure 5.2.x Analysis sequence diagram for UC-xx; message numbers match the scenario table

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Write each step as a declarative sentence first, subject, verb, object, then fill the columns. Tables filled directly produce rows with no subject.

One message per table row, same numbers. Use the four translation cases from Lesson 14; the diagram is a conversion of the table, not a new drawing.

Messages are verb phrases, not operation signatures. "asks for the routes calling at the stop" is analysis; "getRoutes(stopId: int): List<Route>" is Chapter 9 and is not credited here.

Every object on the diagram is a class in 5.1 or an interaction object you name and explain in one line. One object is clearly in control of the interaction.

Budget: one to one and a half pages per use case.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

Step 2 · Subject: LotFinder · Action: asks for the lots the permit allows · Data: permit type, destination · Object acted upon: PermitRules.

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)		
K2 Every object on a sequence diagram is a domain class or a declared interaction object		
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row		
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row		
K5 Every analyzed use case traces to at least one functional requirement		
K6 The last messages of each diagram make the use case postcondition true		

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Run the six checks across the work of different authors; they are designed to catch the seams between people. A failed check that you found and fixed is more convincing than a column of passes. Budget: a quarter of a page.

6 High-Level Design (Architecture)

Chapter 8. The rule now reverses: a high-level design that does not say where things run, what they are built from and what that choice cost has not been done.

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01						QR-
QAS-02						QR-
QAS-03						QR-

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

At least three scenarios, preferably covering more than one quality attribute (for example performance, availability, security, modifiability, usability). Derive them from your P1 quality requirements and say which.

The response measure is the part teams leave out, and it is the only part that lets anyone tell whether the architecture worked. A scenario without a number is an opinion.

Below the table, list in two or three lines the constraints (CON IDs) that shape the architecture.

Budget: half a page.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

QAS-01 · Performance · Stimulus: 400 drivers request lot status during the 7:45 to 8:15 AM peak · Artifact: the occupancy service · Response: returns current open-space counts · Measure: within 2 s for 95% of requests · From QR-02.

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
			Chosen
			Rejected
			Rejected

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Name the style (or combination) you chose and at least two you rejected, using the vocabulary from Lessons 15 and 16: layered (N-tier), client-server, repository, model-view-controller, pipe-and-filter, event-driven, service-oriented or microservices.

One paragraph below the table explains the choice in terms of the scenarios in 6.1. "We are most comfortable with it" is not a driver. ADR-01 in section 7 records this decision formally.

Budget: half a page.

6.3 Component view

Insert diagram here, then replace this frame with the image.

Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Point at any box and say which use case, which quality scenario or which decision put it there. A box with no answer cannot be credited.

Show external systems outside the boundary. Name interfaces by what they offer ("OccupancyQuery"), not by operation signatures, which are Chapter 9.

Budget: one page including the diagram.

6.4 Deployment view

Insert diagram here, then replace this frame with the image.

Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Every component in 6.3 is deployed somewhere in 6.4. Every communication path names its protocol. Mark the trust boundaries: where data crosses from a zone you control into one you do not.

If you use a cloud service, say which responsibilities stay with your team under the shared-responsibility model.

Budget: one page including the diagram.

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Every class and interaction object from section 5 lands in exactly one component, or the note says why it was split, merged or left to an external system. This table is how the reader sees that the architecture realizes the analysis rather than replacing it. Budget: a quarter of a page.

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns		
Information hiding		
Low coupling		
High cohesion		
Design for change		
Keep it simple (KISS)		

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
>
> One line per principle, pointing at something concrete. A principle you cannot point at in your own design should be left blank and explained in one sentence, not filled with a definition from the slides. Budget: a third of a page.

7 Design Rationale

7.1 Architecture decision records

At least three. ADR-01 records the style decision from 6.2. Use the format you practiced in the Lesson 16 workshop, written about your own project. Copy the block for each decision.

ADR-xx [Decision title]

Status	Accepted / Superseded by ADR-
Date and owner	
Context (drivers)	Which QAS, use cases and constraints force a decision here
Options considered	At least two, each in one line
Decision	
Consequences	Name the quality it favors, the move, and the cost you accept
Evidence	Spike result (section 8), measurement, documentation, or "none yet" with an open question
Related	ADR-, QAS-, R- (risk), CH- (change)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

A decision without a cost is a preference. If you cannot say what you gave up, you have not finished the record.

Options must be real alternatives the team discussed, not a straw man added to fill the row.

Budget: about a third of a page per ADR.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

ADR-02 · Push occupancy changes to clients rather than polling · Context: QAS-01, CON-02 (campus Wi-Fi limits) · Options: client polling every 10 s; server push over a persistent connection · Decision: server push · Consequences: favors performance and freshness; costs connection management on the occupancy service and a fallback when push fails (A2 in UC-02) · Evidence: spike S-01.

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
			ADR-
			ADR-
			ADR-

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

One row per real trade-off. This is the table a reviewer reads first. Budget: a quarter of a page.

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01
Risk or decision it addresses	R- / ADR- / QAS-
Question (answerable yes or no)	
Set-up	Stack, data, what was simulated; folder in the repository
How it was measured	
Result (numbers)	
What it changed or confirmed	Which ADR, QAS or requirement
Where to see it	Commit hash, /spikes/S-01/, screenshot or log file

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Build the smallest thing that answers your riskiest architectural question, run it, and measure it. Time-box it to a few hours. It may be thrown away afterwards; it is not the product.

Good spikes answer questions such as: can we read the external data source, and is it fresh to within 60 seconds? Does the chosen hosting meet QAS-01 under a simulated peak? Does the authentication provider fit our trust boundary?

Never commit passwords, API keys or other credentials; keep them in local configuration excluded from the repository.

The code, the measurement and a short README are committed to /spikes before the deadline. A spike described but not in the repository is not evidence.

Budget: half a page.

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01					
FR-02					
QR-01					
CON-01					

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Extend the P1 matrix one column at a time: from each requirement to the use case, to the analysis classes that realize it, to the component that will own it, to the decision or scenario that shaped it, and to whatever evidence exists (spike, measurement).

Requirements realized by the use cases you analyzed need the full chain. Every other MUST requirement needs at least its component. Withdrawn requirements stay in the matrix, marked as such.

Budget: one page.

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component		
Components with no requirement, QAS or ADR		
Use cases not analyzed (and why that is acceptable)		

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

An empty gap table is believable only if the matrix above is complete. Say what each gap means and what you did. Budget: a quarter of a page.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
>
> Keep your P1 text and record only what changed. If roles moved between members, say so here and in 0.2.

10.2 Plan to P3

Milestone	Date	Owner	Done when

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1			
R2			
R6 (new)			

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
>
> Close or downgrade P1 risks with a reason. Add the architectural risks your design created, and point to the spike or ADR that addresses each.

10.4 Repository

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
>
> State the repository URL, the folder layout (/docs/p2, /docs/p2/diagrams, /spikes), your branch and pull-request rule, and confirm that the tag p2 points at the commit containing this PDF.
>
> **Access:** the repository is public, or the instructor account announced on Canvas has read access before the deadline. A repository that cannot be opened at grading time cannot earn repository or contribution credit.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
>
> Declare every use, including code assistants used in the spike. Commit messages carrying an AI co-author line must be matched by a row here. Using AI is permitted; using it without declaring it is not. Write "None" if no tool was used.

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q					
Q					

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
>
> Close the P1 questions you answered and say how. Add the questions analysis and design raised: a step whose triviality was unclear, a scenario with no agreed response measure. If no questions remain open, say how the earlier ones were resolved.

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date

How P2 Is Marked

Delete this page and the next before submitting. 60 points: 48 for the shared team artifact (80%) and 12 for individual contribution evidence (20%).

Criterion	Points	What full marks look like
Baseline, change control, plan and repository (sections 0.2, 1 to 3, 10 to 12)	6	Every point of the P1 feedback is answered in 0.2. Requirement IDs are preserved and withdrawn items marked. The plan to P3, risks, AI-use declaration, open questions and signed v1.0 baseline are up to date. The repository is accessible and, before the deadline, holds this PDF, the diagram sources and the spike, with the tag p2 on that commit.
Use-case analysis (section 4)	8	At least three use cases expanded turn by turn with step (0), one turn per row, an acknowledgment at the end, real alternate flows, and no technology. Preconditions are true assumptions, not hidden alternate flows. The self-review log records what each check found, including any defects fixed.
Domain model and object interaction (section 5)	10	A conceptual domain model with attributes and multiplicities that contains every item the actor columns mention. At least two use cases carried through non-trivial steps, a five-column scenario table and an analysis sequence diagram whose messages match the table row for row. The six cross-checks are run honestly.
High-level design (section 6)	10	At least three quality-attribute scenarios with response measures. A named style chosen over at least two alternatives for reasons tied to those scenarios. A component view in which every component answers to a use case, scenario or decision; a UML deployment view with protocols and trust boundaries; every analysis object mapped to a component; design principles pointed out in the design itself.
Design rationale (section 7)	6	At least three decision records, each with real alternatives, a decision and a stated cost. The trade-off summary agrees with them.
Prototype evidence (section 8)	4	One spike that answers a real architectural question with a measured result, committed with its README before the deadline, and linked to the decision it changed or confirmed.
Traceability (section 9)	4	Every requirement realized by the analyzed use cases is traced through analysis classes to a component, and to the scenario or decision that shaped it where one exists. Every other MUST requirement reaches at least a component. Gaps are named, explained and acted on.
Document quality	up to −3	Deducted from the team points, not awarded: guidance, example or warning boxes left in, placeholders, sections out of order or renumbered, unreadable diagrams, or a document far beyond its length budget.

Criterion	Points	What full marks look like
Individual contribution evidence (section 0.1)	12 each	12: owns specific sections or artifacts, with own commits or pull requests before the deadline and at least one review of a teammate's pull request. 9: specific ownership and own commits, but thin or no review evidence. 6: specific claim with indirect evidence only (issues owned, or work uploaded explicitly on the member's behalf). 4: specific claim with no repository evidence. 2: vague or duplicated claim. 0: no credited work. A member with no credited work at all (absent from the contribution record and with no commits) may also lose the team portion, because the syllabus does not allow credit for work completed by others. This applies to individual non-participation only, never to a whole team.

WHERE P1 MARKS WERE LOST (DELETE THIS BOX BEFORE SUBMITTING)

Repositories that were empty, unreachable, or held a single upload from one account at the deadline. Repository and contribution credit could not be given, however good the document was.

Contribution records that cited commits, pull requests or files that were not in the repository. Everything cited is checked.

Guidance boxes, placeholders and unconfirmed draft text left in the submitted document.

Several members uploading different versions to Canvas. One member submits. If more than one file is uploaded, the latest upload is the one graded.

Quality requirements with no number in them, and requirements whose source was still "pending".

Work committed after the deadline. It is not counted, even a few minutes after.